

COFFEE SHOP SALES ANALYTICS

An End-to-End PySpark Big Data Project

Medallion Architecture • Star Schema • Window Functions • Broadcast Joins

Prepared by Youssef — Big Data Engineering Interview Prep

Dataset: Coffee_Shop_Dataset.csv — 149,116 transactions · 3 NYC stores · Jan–Jun 2023

1 · Project Overview

This project builds a complete PySpark analytics pipeline on a real coffee-shop point-of-sale dataset: **149,116 transaction-level rows** spanning **3 New York City stores** (Astoria, Lower Manhattan, Hell's Kitchen) from January through June 2023. The goal is twofold: (1) demonstrate a production-style Spark pipeline using the **medallion architecture** (Bronze → Silver → Gold), and (2) answer concrete business questions using core PySpark analytics techniques — window functions, broadcast joins, and aggregations — that come up constantly in Big Data engineering interviews.

149,116
Rows Processed

3
Stores

98
Product Types

\$820K
Total Revenue

Pipeline Architecture

The project follows the **medallion (multi-hop) architecture** — the same pattern used across this interview-prep series in both SQL and PySpark form:

Layer	Purpose	Key Operations
Bronze	Raw ingestion, nothing dropped	Explicit schema, header parsing
Silver	Cleaned, typed, validated	Date/time parsing, de-dup, data-quality checks
Gold	Analytics-ready star schema	dim_date, dim_store, dim_product, fact_sales + KPIs

Deliverables

- **script.py** — standalone, fully commented PySpark script that runs the whole pipeline end-to-end.
- **Coffee_Shop_PySpark_Analytics.ipynb** — the same pipeline as an executed Jupyter notebook, with all tables and charts rendered inline.
- This PDF — a walkthrough of every stage and every analytical question, with the resulting charts.

2 · Bronze Layer — Raw Ingestion

The raw CSV is read with an **explicit schema** instead of `inferSchema=True`. Schema inference forces Spark to do an extra pass over the file just to guess column types — and it can guess wrong. Declaring the schema up front is both faster and safer, which is why it's standard practice in production ingestion jobs.

```
bronze_df = (
    spark.read
    .option("header", True)
    .schema(bronze_schema)    # StructType with 18 explicit fields
    .csv(RAW_CSV_PATH)
)
```

Result: 149,116 raw rows loaded, schema enforced (transaction_id, store_id, product_id as integers; unit_price/Total Bill as doubles; dates/times still as strings — parsed properly in Silver).

3 · Silver Layer — Cleaning & Data Quality

The Silver layer applies the standard cleaning checklist: rename spaced column names to snake_case, parse the raw "6/1/2023" / "11:33:29 AM" strings into real date/timestamp types, trim whitespace from every string column, drop exact duplicates on the natural key, and filter out any structurally invalid rows (non-positive quantity or price).

Data-Quality Cross-Check

Rather than trusting the source Total Bill column, the script **recomputes revenue itself** (transaction_qty × unit_price) and flags any row where the two disagree by more than a cent. This is a classic Silver-layer data-quality gate — and on this dataset it confirms the source data is trustworthy:

Check	Result
Rows before cleaning	149,116
Rows after cleaning (dupes/invalid removed)	149,116 (0 dropped)
Revenue cross-check mismatches	0

4 · Gold Layer — Star Schema

The cleaned data is modeled as a classic **star schema**: one fact table surrounded by conformed dimension tables, the same pattern used throughout this interview-prep series (including the SQL Server version of this same dataset).

Table	Grain	Row Count
dim_date	One row per calendar date	181
dim_store	One row per store	3
dim_product	One row per product SKU	98
fact_sales	One row per transaction line	149,116

date_id uses a yyyyMMdd integer surrogate key — a common, sort-friendly convention for date dimensions. The Gold layer is written to disk as **partitioned Parquet** (fact_sales partitioned by month_num), giving downstream jobs columnar compression and partition pruning on month filters.

5 · Analytics — Window Functions & Broadcast Joins

Every query below joins the fact table to the dimension tables using `F.broadcast()`. Since the dimension tables are tiny (3 stores, 98 products, 181 dates) compared to the ~149K-row fact table, broadcasting ships a full copy of each small table to every executor instead of shuffling the large fact table across the network — the standard optimization for "big fact table, small dimension table" joins.

5a · Revenue by Store & Month — Running Cumulative Total

A window function with an unbounded-preceding frame — `rowsBetween` (`Window.unboundedPreceding`, `Window.currentRow`) — computes each store's running cumulative revenue, the PySpark equivalent of SQL's `SUM(...) OVER (PARTITION BY store ORDER BY month ROWS UNBOUNDED PRECEDING)`.

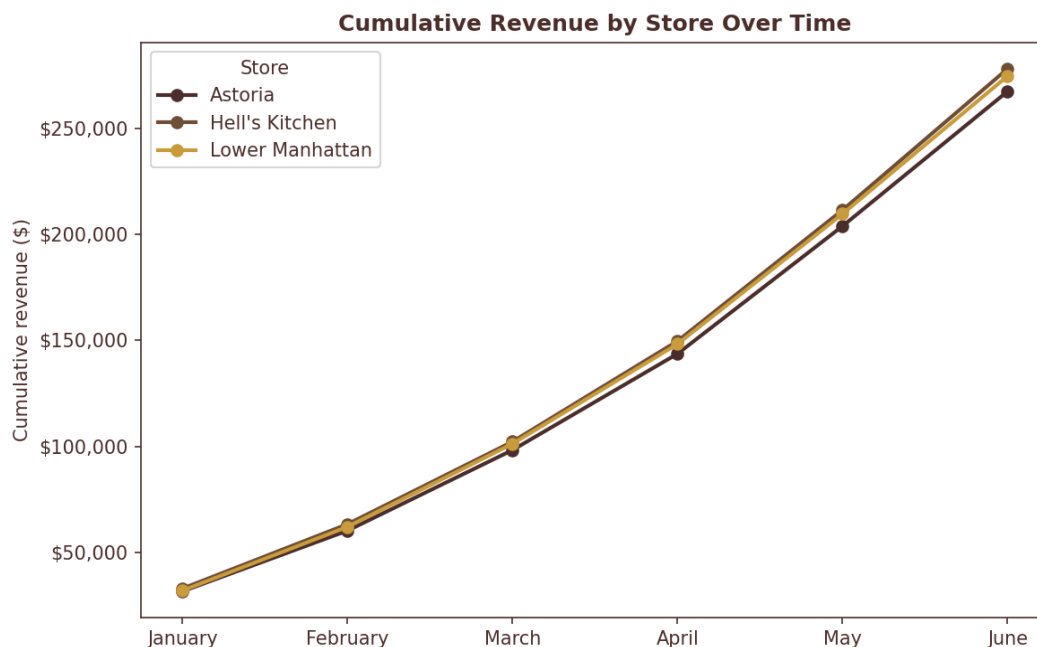


Figure 1 — Cumulative revenue by store, Jan–Jun 2023. All three stores show consistent growth.

5b · Month-over-Month Growth — LAG()

F. `lag()` reaches back to the previous row in an ordered window (here, the previous month's revenue) to compute percentage growth — SQL's `LAG(revenue) OVER (ORDER BY month)` pattern.

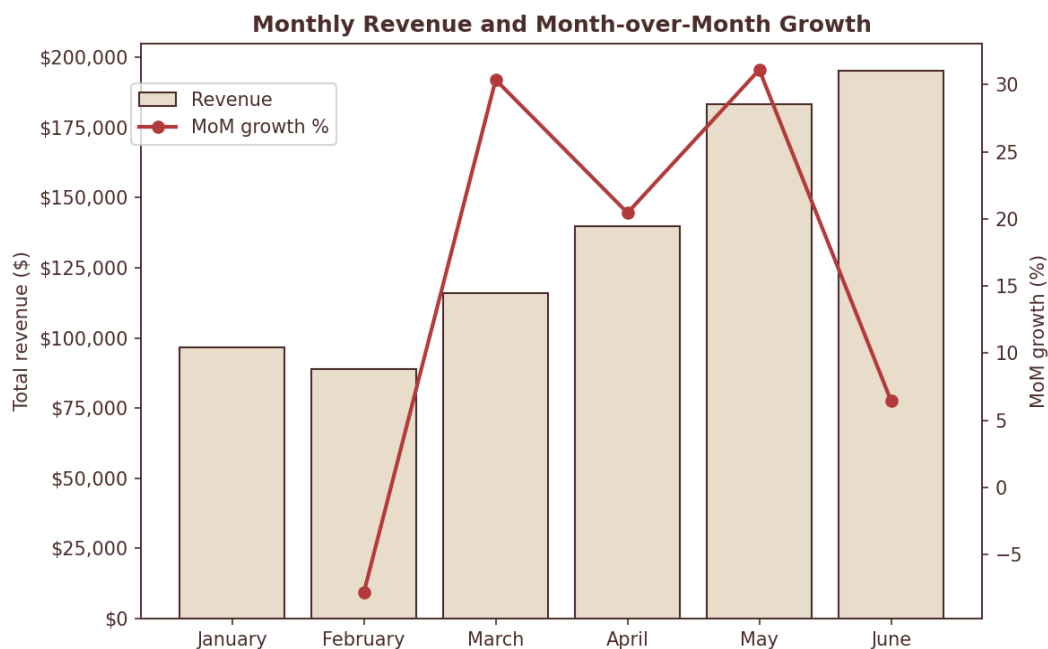


Figure 2 — Revenue grew every month except February (-7.8%); May was the strongest month (+31.1%).

Month	Revenue	MoM Growth
January	\$96,617	—

Month	Revenue	MoM Growth
February	\$89,066	-7.82%
March	\$116,106	+30.36%
April	\$139,851	+20.45%
May	\$183,356	+31.11%
June	\$195,165	+6.44%

5c · Top-3 Product Types per Category — DENSE_RANK()

F . dense_rank () over a window partitioned by category and ordered by revenue ranks products within each category without gaps; filtering to rank ≤ 3 gives the classic "top-N per group" result.

Category	#1 Product	#2 Product	#3 Product
Coffee	Barista Espresso (\$100.3K)	Gourmet brewed (\$70.0K)	Premium brewed (\$38.8K)
Tea	Brewed Chai (\$77.1K)	Brewed Black (\$47.9K)	Brewed Herbal (\$47.5K)
Bakery	Scone (\$76.3K)	Pastry (\$51.3K)	Biscotti (\$39.6K)
Drinking Chocolate	Hot chocolate (\$72.4K)	—	—

5d · Store Performance Ranking — RANK()

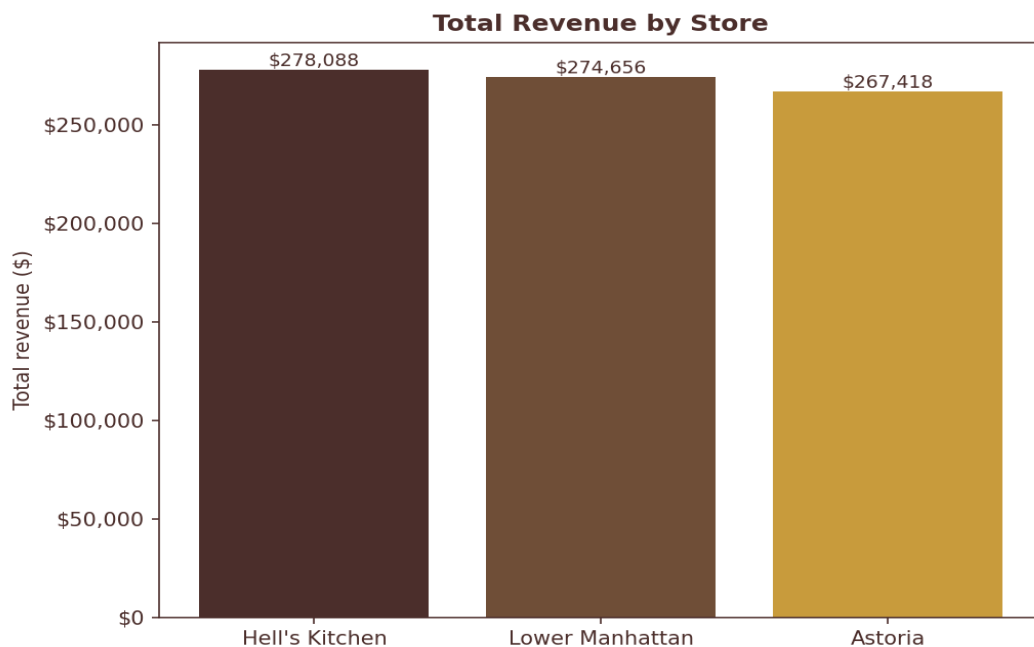


Figure 3 — Hell's Kitchen leads on total revenue; Lower Manhattan has the highest average basket value (\$4.79).

Rank	Store	Total Revenue	Transactions	Avg Basket
1	Hell's Kitchen	\$278,088	60,407	\$4.60
2	Lower Manhattan	\$274,656	57,310	\$4.79
3	Astoria	\$267,418	58,294	\$4.59

5e · Hour-of-Day & Day-of-Week Demand Patterns

Simple groupBy aggregations reveal peak ordering times — useful input for staffing decisions.

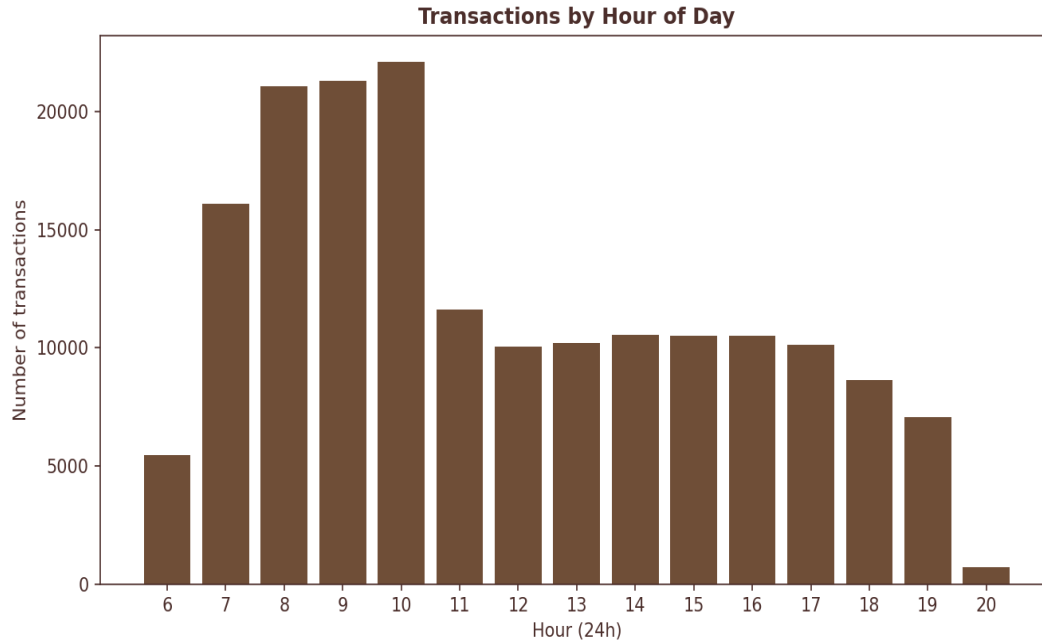


Figure 4 — Demand peaks in the mid-morning commuter hours (around 8-10 AM).

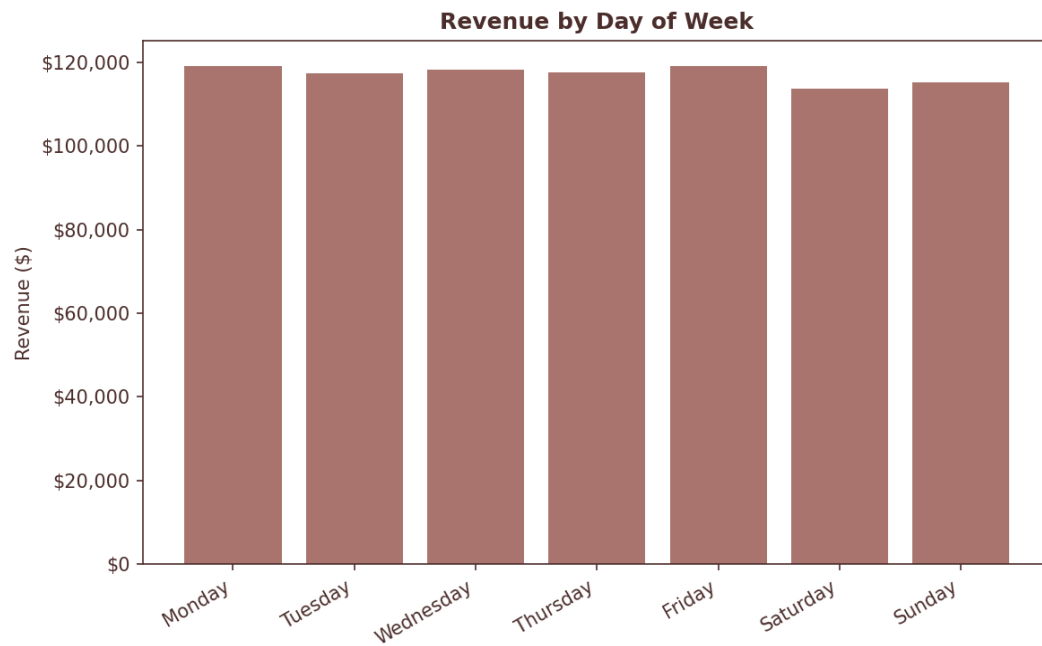


Figure 5 — Revenue is fairly even across weekdays with a slight dip on Sunday.

5f · Revenue Share by Product Category

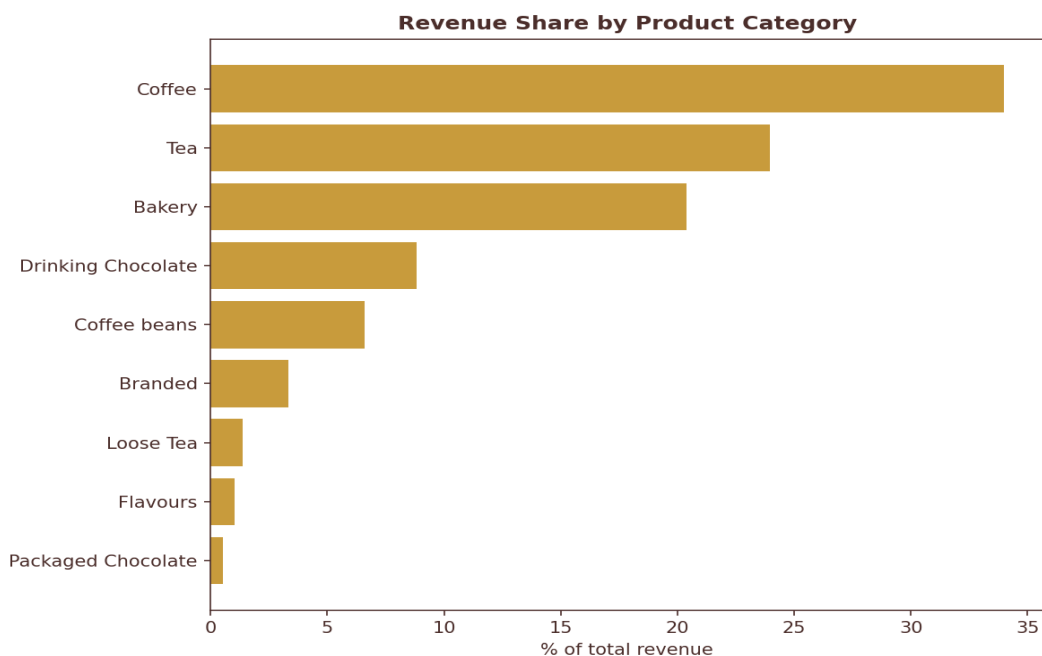


Figure 6 — Coffee (34%), Tea (24%) and Bakery (20%) together drive ~78% of total revenue.

Category	Revenue	% of Total
Coffee	\$278,855	34.0%
Tea	\$196,406	23.9%
Bakery	\$167,183	20.4%
Drinking Chocolate	\$72,416	8.8%
Coffee beans	\$54,058	6.6%
Branded / Other	\$41,244	5.0%

5g · Size Mix Within Each Category

A self-join against per-category totals (also broadcast, since it's tiny) converts raw unit counts into within-category percentages. **Key finding:** Coffee sells fairly evenly across Small (23%), Regular (28%) and Large (27%), while Tea skews toward Regular (51%) over Large (49%). Bakery and Branded items carry no size dimension.

6 · Key Takeaways & Techniques Demonstrated

Business Findings

- Revenue grew nearly every month (Jan→Jun), with a dip only in February; May was the strongest growth month at +31%.

- Coffee, Tea and Bakery drive ~78% of total revenue — a small set of "hero" product types (Espresso, Chai, Scone) dominate within each category.
- All three stores perform within ~4% of each other on revenue — Hell's Kitchen leads on volume, Lower Manhattan on basket size.
- Demand is concentrated in the mid-morning commuter window, a useful signal for staffing.

PySpark Techniques Demonstrated

Technique	Where Used
Explicit schema (StructType)	Bronze ingestion
to_date / to_timestamp parsing	Silver cleaning
dropDuplicates, data-quality flag	Silver cleaning
Star schema design + partitioned Parquet	Gold layer
Broadcast join (F.broadcast)	All analytics joins
Window: rowsBetween (running total)	5a
Window: LAG (month-over-month growth)	5b
Window: DENSE_RANK (top-N per group)	5c
Window: RANK (store ranking)	5d
groupBy aggregations	5e, 5f, 5g

Full code: *script.py* (standalone) and *Coffee_Shop_PySpark_Analytics.ipynb* (executed notebook, same pipeline, outputs rendered inline).